

Pontificia Universidad Católica de Chile

Departamento de Ingeniería Mecánica y Metalúrgica



Elementos Finitos en Diseño Mecánico: Tarea 1

30 de Agosto de 2026 - Profesor Diego J. Celentano

Estudiantes: Ignacio Chacón.

Declaración de IA

Declaración de uso de inteligencia artificial

Para el desarrollo de esta tarea se utilizó ChatGPT como herramienta de apoyo para la revisión conceptual, organización del informe y revisión de código en Python y L^AT_EX. Asimismo, se utilizó para proponer formas de presentación y visualización de los resultados, usando las herramientas de programación para realizar gráficos y tablas.

Problema 1: Barra de sección variable

Enunciado y datos del problema

La barra de acero ($E = 206$ GPa y $S_y = 300$ MPa) en posición vertical de la Figura 1 ($L = 10$ m) tiene sección transversal variable,

$$A(z) = A_0 e^{-z/L}, \quad (1)$$

con $A_0 = 1600$ mm², y está sometida a una fuerza axial en su extremo libre ($P = 3$ kN).

De acuerdo con la teoría de barras, se pide:

- a) Obtener la expresión analítica de los campos de tensión y desplazamiento axial.
- b) Obtener la solución por MEF de dichos campos utilizando discretizaciones con 1, 2 y 4 elementos finitos de dos nodos de área constante en cada uno de ellos.
- c) Comparar y discutir las soluciones obtenidas en a) y b).

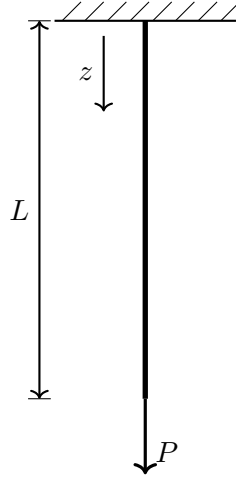


Figura 1. Barra vertical de sección transversal variable sometida a una carga axial P .

Solución analítica

Antes de desarrollar el modelo mediante el Método de los Elementos Finitos, se obtiene la solución analítica del problema. Esta solución permite conocer la distribución exacta de desplazamientos y tensiones a lo largo de la barra, y será utilizada posteriormente como referencia para evaluar la precisión de las distintas discretizaciones empleadas.

Fuerza interna axial

Considerando un elemento diferencial de la barra y suponiendo que no existen cargas distribuidas en la dirección longitudinal, la ecuación de equilibrio axial está dada por

$$\frac{dN}{dz} = 0, \quad (2)$$

donde $N(z)$ corresponde a la fuerza axial interna.

La ecuación anterior indica que la fuerza interna permanece constante a lo largo de toda la barra. Aplicando la condición de borde en el extremo cargado,

$$N(L) = P, \quad (3)$$

se obtiene

$$\boxed{N(z) = P}. \quad (4)$$

Por lo tanto, a pesar de que la sección transversal varía a lo largo de la barra, la fuerza

axial transmitida por cada sección es la misma.

Campo de tensiones

La tensión normal se relaciona con la fuerza axial mediante

$$\sigma(z) = \frac{N(z)}{A(z)}. \quad (5)$$

Utilizando $N(z) = P$ y la expresión correspondiente al área,

$$\sigma(z) = \frac{P}{A_0 e^{-z/L}}, \quad (6)$$

por lo que el campo de tensiones exacto queda dado por

$$\boxed{\sigma(z) = \frac{P}{A_0} e^{z/L}}. \quad (7)$$

Esta expresión muestra que la tensión aumenta a medida que disminuye el área transversal. Debido a que la fuerza axial es constante, la sección de menor área debe transmitir la misma carga y, por lo tanto, se encuentra sometida a una tensión mayor.

En el extremo fijo, $z = 0$, se obtiene

$$\sigma(0) = \frac{P}{A_0} = \frac{3000}{1600} = 1.875 \text{ MPa}. \quad (8)$$

Mientras que en el extremo cargado, $z = L$,

$$\sigma(L) = \frac{P}{A_0} e, \quad (9)$$

obteniéndose

$$\boxed{\sigma(L) = 5.097 \text{ MPa}}. \quad (10)$$

Por lo tanto, la máxima tensión se alcanza en el extremo de menor sección transversal.

Campo de deformaciones

Para un material lineal elástico, la relación constitutiva uniaxial corresponde a la ley de Hooke,

$$\sigma(z) = E\varepsilon(z). \quad (11)$$

De esta relación se obtiene

$$\varepsilon(z) = \frac{\sigma(z)}{E}. \quad (12)$$

Sustituyendo el campo de tensiones obtenido anteriormente,

$$\boxed{\varepsilon(z) = \frac{P}{EA_0} e^{z/L}}. \quad (13)$$

Al igual que la tensión, la deformación axial aumenta hacia el extremo de menor sección, ya que ambas magnitudes se encuentran relacionadas directamente mediante el módulo de Young.

Campo de desplazamientos

La deformación longitudinal también puede expresarse en términos del desplazamiento axial $w(z)$ como

$$\varepsilon(z) = \frac{dw}{dz}. \quad (14)$$

Utilizando la expresión obtenida en la Ec. (13),

$$\frac{dw}{dz} = \frac{P}{EA_0} e^{z/L}. \quad (15)$$

Integrando respecto de z ,

$$w(z) = \frac{P}{EA_0} \int e^{z/L} dz, \quad (16)$$

y considerando que

$$\int e^{z/L} dz = L e^{z/L}, \quad (17)$$

se obtiene

$$w(z) = \frac{PL}{EA_0} e^{z/L} + C, \quad (18)$$

donde C es una constante de integración.

Debido a que el extremo ubicado en $z = 0$ se encuentra empotrado, su desplazamiento axial es nulo,

$$w(0) = 0. \quad (19)$$

Por lo tanto,

$$0 = \frac{PL}{EA_0} + C, \quad (20)$$

de donde

$$C = -\frac{PL}{EA_0}. \quad (21)$$

Finalmente, el campo de desplazamientos exacto de la barra queda dado por

$$\boxed{w(z) = \frac{PL}{EA_0} \left(e^{z/L} - 1 \right)}. \quad (22)$$

El desplazamiento aumenta de forma no lineal a lo largo de la barra. Esto se debe a que la rigidez axial local,

$$EA(z), \quad (23)$$

disminuye a medida que aumenta z , provocando que una misma fuerza axial genere deformaciones progresivamente mayores.

Desplazamiento máximo

El máximo desplazamiento ocurre en el extremo libre de la barra, es decir, para $z = L$. Evaluando la Ec. (22),

$$w(L) = \frac{PL}{EA_0} (e - 1). \quad (24)$$

Sustituyendo los valores correspondientes,

$$w(L) = \frac{(3000)(10000)}{(206000)(1600)} (e - 1), \quad (25)$$

obteniéndose

$$\boxed{w(L) = 0.1564 \text{ mm}}. \quad (26)$$

Formulación mediante el Método de Elementos Finitos

Para obtener una aproximación numérica de la solución, la barra se discretiza mediante elementos unidimensionales de dos nodos. Se consideran discretizaciones de 1, 2 y 4 elementos, manteniendo en todos los casos las mismas propiedades del material y condiciones de borde.

La implementación computacional se organizó mediante distintas funciones, cada una asociada a una etapa del Método de los Elementos Finitos. De esta forma, el procedimiento seguido corresponde a la determinación de la geometría y propiedades de cada elemento, construcción de sus matrices de rigidez, ensamblaje de la matriz global, aplicación de las condiciones de borde, resolución de los desplazamientos nodales y posterior cálculo de deformaciones y tensiones.

Discretización y propiedades de los elementos

Para una discretización de n_e elementos, el número de nodos está dado por

$$n_n = n_e + 1, \quad (27)$$

mientras que, debido a que los elementos poseen igual longitud,

$$L_e = \frac{L}{n_e}. \quad (28)$$

Debido a que el área de la barra varía continuamente según

$$A(z) = A_0 e^{-z/L}, \quad (29)$$

es necesario asignar un valor de área a cada elemento para construir su matriz de rigidez. En este trabajo se aproxima el área de cada elemento mediante su valor en el punto medio.

Computacionalmente, esta operación se realiza mediante la función `centros_nodos`, la cual determina las posiciones utilizadas para evaluar el área de cada elemento. Posteriormente se calcula

$$A_e = A_0 e^{-z_e/L}, \quad (30)$$

donde z_e corresponde a la posición central del elemento considerado.

En el código, este procedimiento se implementa mediante

```
zc_nodos = centros_nodos(nodos, L)
area_nodo = a_0 * np.exp(-zc_nodos/L)
```

El primer valor almacenado en `zc_nodos` corresponde a $z = 0$, mientras que las posiciones posteriores corresponden a los centros de los elementos. Por esta razón, al calcular la rigidez se utiliza el valor `area_nodo[i+1]` para cada elemento.

Esta aproximación implica que, aunque el área real cambia continuamente, dentro de cada elemento se considera un área constante. Al aumentar el número de elementos, esta representación por tramos permite aproximar progresivamente mejor la variación real de la sección transversal.

Matriz de rigidez elemental

Para un elemento de barra unidimensional de dos nodos sometido únicamente a carga axial, el vector de desplazamientos elementales se define como

$$\mathbf{u}^{(e)} = \begin{bmatrix} w_i \\ w_j \end{bmatrix}, \quad (31)$$

donde w_i y w_j corresponden a los desplazamientos axiales de sus nodos extremos.

La matriz de rigidez elemental está dada por

$$\mathbf{K}^{(e)} = \frac{EA_e}{L_e} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}. \quad (32)$$

El término

$$k_e = \frac{EA_e}{L_e} \quad (33)$$

representa la rigidez axial del elemento. De esta expresión se observa que un aumento del módulo de Young o del área transversal incrementa la rigidez, mientras que una mayor longitud del elemento la disminuye.

En la implementación, los valores k_e se calculan mediante la función `calcular_vector_k`:

```
vector_k[i] = E * area_nodo[i + 1] / (L/elementos)
```

De esta manera se obtiene un valor de rigidez independiente para cada elemento, ya que el área transversal utilizada depende de su posición a lo largo de la barra.

Posteriormente, para cada elemento se construye la matriz de la Ec. (32) mediante

```
Ki = vector_k[i] * np.array([[1, -1],
                              [-1, 1]])
```


Ensamblaje de la matriz global de rigidez

Una vez obtenidas las matrices elementales, estas deben ensamblarse para representar el comportamiento de la barra completa. La matriz global de rigidez posee dimensiones

$$n_n \times n_n, \quad (34)$$

ya que cada nodo presenta un único grado de libertad correspondiente al desplazamiento axial.

Inicialmente se define una matriz global nula,

$$\mathbf{K} = \mathbf{0}, \quad (35)$$

y la contribución de cada elemento se incorpora en las posiciones asociadas a sus nodos.

En el código, el ensamblaje se realiza en la función `matriz_k` mediante

```
K[i:i+2, i:i+2] += Ki
```

El operador `+=` es importante debido a que dos elementos consecutivos comparten un nodo. Por lo tanto, la rigidez correspondiente a dicho grado de libertad debe considerar la contribución de ambos elementos.

Por ejemplo, para una discretización de dos elementos, las matrices elementales corresponden a

$$\mathbf{K}^{(1)} = k_1 \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}, \quad (36)$$

y

$$\mathbf{K}^{(2)} = k_2 \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}. \quad (37)$$

Al ensamblarlas se obtiene

$$\mathbf{K} = \begin{bmatrix} k_1 & -k_1 & 0 \\ -k_1 & k_1 + k_2 & -k_2 \\ 0 & -k_2 & k_2 \end{bmatrix}. \quad (38)$$

El término $k_1 + k_2$ en el nodo central representa precisamente la suma de las contribuciones de los dos elementos que se encuentran conectados a dicho nodo.

Vector de cargas y condiciones de borde

Una vez construida la matriz global, el sistema de ecuaciones del Método de los Elementos Finitos puede expresarse como

$$\mathbf{K}\mathbf{U} = \mathbf{F}, \quad (39)$$

donde $\mathbf{U}$ contiene los desplazamientos nodales y $\mathbf{F}$ corresponde al vector de fuerzas externas.

Como la única carga aplicada corresponde a la fuerza axial P en el último nodo de la barra, el vector de fuerzas tiene la forma

$$\mathbf{F} = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ P \end{bmatrix}. \quad (40)$$

Esto se implementa computacionalmente mediante

```
F = np.zeros(nodos)
F[-1] = P
```

Por otra parte, el extremo ubicado en $z = 0$ se encuentra fijo, por lo que se impone la condición esencial

$$w(0) = 0. \quad (41)$$

Dado que este desplazamiento ya es conocido, el primer grado de libertad no necesita ser resuelto. Por este motivo se obtiene un sistema reducido eliminando la primera fila y columna asociadas al desplazamiento restringido:

```
Kr = K[1:, 1:]
Fr = F[1:]
```

El sistema reducido queda entonces expresado como

$$\mathbf{K}_r \mathbf{U}_r = \mathbf{F}_r. \quad (42)$$

Resolución de los desplazamientos nodales

Los desplazamientos desconocidos se calculan resolviendo el sistema lineal anterior. En la función `resolver_sistema` se utiliza

```
Ur = np.linalg.solve(Kr, Fr)
```

obteniéndose el vector de desplazamientos de los grados de libertad no restringidos. Posteriormente se reconstruye el vector completo,

$$\mathbf{U} = \begin{bmatrix} 0 \\ w_1 \\ \vdots \\ w_{n_n-1} \end{bmatrix}, \quad (43)$$

incorporando explícitamente el desplazamiento nulo del primer nodo. En el código esto se realiza mediante

```
U = np.zeros(nodos)
U[1:] = Ur
```

Los valores obtenidos corresponden a los desplazamientos nodales aproximados por el MEF. Entre dos nodos consecutivos, el elemento de barra de dos nodos representa el desplazamiento mediante una interpolación lineal.

Cálculo de deformaciones

Una vez conocidos los desplazamientos nodales, se calcula la deformación de cada elemento. Para un elemento lineal de longitud L_e ,

$$\varepsilon_e = \frac{w_j - w_i}{L_e}. \quad (44)$$

Debido a que se utiliza una interpolación lineal para el desplazamiento, la deformación obtenida es constante dentro de cada elemento.

La función `calcular_deformaciones` implementa directamente esta expresión:

```
deformaciones[i] = (U[i + 1] - U[i]) / Le
```

donde los valores $U[i]$ y $U[i+1]$ corresponden a los desplazamientos de los nodos extremos del elemento.

Cálculo de tensiones

Finalmente, considerando un comportamiento lineal elástico, las tensiones de cada elemento se calculan mediante la ley de Hooke,

$$\sigma_e = E\varepsilon_e. \quad (45)$$

Esta operación se realiza mediante la función `calcular_tensiones`:

```
tensiones = E * deformaciones
```

Como la deformación es constante dentro de cada elemento, la tensión obtenida mediante esta formulación también es constante en cada elemento. Por esta razón, posteriormente el campo de tensiones del MEF se representa mediante valores constantes por tramos, mientras que el campo analítico varía continuamente a lo largo de la barra.

Resultados

A partir de la formulación presentada anteriormente, se resolvió el problema utilizando discretizaciones de 1, 2 y 4 elementos. Para cada caso se obtuvieron los desplazamientos nodales, deformaciones y tensiones elementales. Los resultados se comparan con la solución analítica obtenida previamente.

Campo de desplazamientos

La Figura 2 presenta el campo de desplazamientos obtenido mediante la solución analítica y las tres discretizaciones utilizadas en el modelo de elementos finitos.

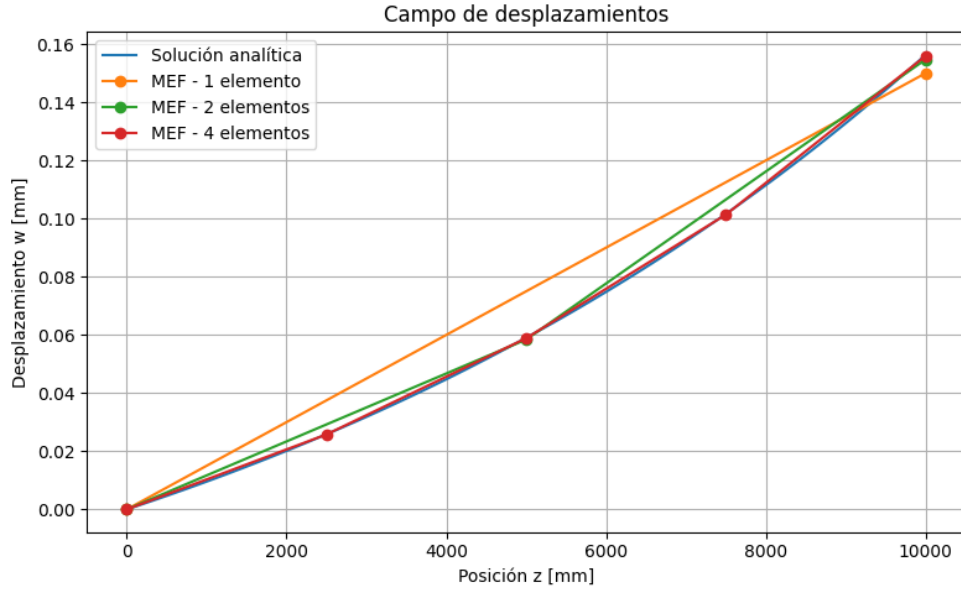


Figura 2. Comparación del campo de desplazamientos obtenido analíticamente y mediante discretizaciones de 1, 2 y 4 elementos.

En la solución mediante elementos finitos, el desplazamiento se encuentra definido en los nodos y se interpola linealmente dentro de cada elemento. Por esta razón, la aproximación obtenida corresponde a segmentos rectos entre nodos, mientras que la solución analítica presenta una variación continua y no lineal.

Los desplazamientos obtenidos en el extremo cargado se resumen en la Tabla 1. El error porcentual se calcula respecto del desplazamiento analítico mediante

$$\text{Error} = \left| \frac{w_{\text{MEF}} - w_{\text{analítico}}}{w_{\text{analítico}}} \right| 100. \quad (46)$$

Tabla 1: Desplazamiento en el extremo cargado para las distintas discretizaciones.

Método	$w(L)$ [mm]	Error [%]
Solución analítica	0.1564	–
MEF – 1 elemento	0.1501	4.05
MEF – 2 elementos	0.1548	1.03
MEF – 4 elementos	0.1560	0.26

Campo de tensiones

La comparación entre el campo de tensiones analítico y las soluciones obtenidas mediante elementos finitos se presenta en la Figura 3.

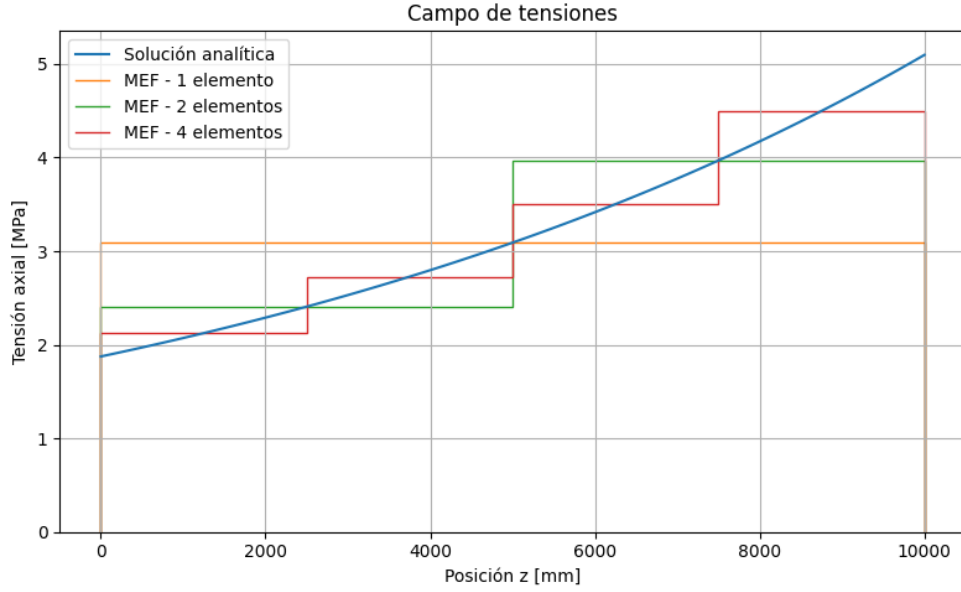


Figura 3. Comparación del campo de tensiones obtenido analíticamente y mediante discretizaciones de 1, 2 y 4 elementos.

A diferencia del desplazamiento, la tensión obtenida mediante cada elemento es constante en su interior. Esto se debe a la aproximación lineal utilizada y explicada anteriormente.

Los valores de tensión obtenidos para cada elemento se presentan en la Tabla 2.

Tabla 2: Tensiones obtenidas mediante las distintas discretizaciones.

Discretización	Elemento	σ_e [MPa]
1 elemento	1	3.091
2 elementos	1	2.408
	2	3.969
4 elementos	1	2.125
	2	2.728
	3	3.503
	4	4.498

La solución analítica presenta una tensión que varía continuamente desde

$$\sigma(0) = 1.875 \text{ MPa} \quad (47)$$

hasta un valor máximo de

$$\sigma(L) = 5.097 \text{ MPa}. \quad (48)$$

En cambio, las soluciones mediante elementos finitos representan este campo mediante valores constantes por tramos, uno por cada elemento.

Análisis y discusión

Los resultados obtenidos muestran una convergencia progresiva de la solución mediante elementos finitos hacia la solución analítica a medida que se refina la discretización. Este comportamiento se observa tanto en el campo de desplazamientos como en el campo de tensiones, aunque la forma en que se aproximan ambas variables es diferente debido a la formulación empleada.

Convergencia del campo de desplazamientos

Como se observa en la Figura 2, la discretización de un solo elemento representa el desplazamiento mediante una única recta entre los extremos de la barra. Esta aproximación reproduce correctamente las condiciones de borde, pero no permite representar la curvatura del campo analítico de desplazamientos.

Esta diferencia se debe a que la solución exacta está dada por

$$w(z) = \frac{PL}{EA_0} \left(e^{z/L} - 1 \right), \quad (49)$$

por lo que el desplazamiento presenta un comportamiento exponencial. En cambio, el elemento de barra de dos nodos utilizado en el modelo interpola linealmente los desplazamientos dentro de cada elemento.

Al aumentar la discretización a dos y cuatro elementos, la aproximación queda formada por un mayor número de segmentos lineales. De esta manera, cada elemento debe representar una porción menor de la curva analítica y la solución numérica se aproxima progresivamente a ella.

Esta convergencia también puede observarse a partir del desplazamiento en el extremo cargado. Los errores obtenidos para uno, dos y cuatro elementos fueron, respectivamente,

$$4.05\%, \quad 1.03\%, \quad 0.26\%. \quad (50)$$

Al duplicar el número de elementos, el error disminuye aproximadamente a una cuarta parte del valor anterior. Por lo tanto, para las discretizaciones estudiadas se observa una rápida convergencia del desplazamiento hacia la solución analítica.

El modelo de cuatro elementos entrega

$$w_{\text{MEF}}(L) = 0.1560 \text{ mm}, \quad (51)$$

mientras que la solución analítica corresponde a

$$w_{\text{analítico}}(L) = 0.1564 \text{ mm}. \quad (52)$$

La pequeña diferencia entre ambos resultados indica que, para el desplazamiento global de la barra, una discretización de cuatro elementos es suficiente para obtener una buena aproximación en este problema.

Comportamiento del campo de tensiones

El comportamiento del campo de tensiones mostrado en la Figura 3 es diferente al observado para los desplazamientos. La solución analítica presenta un aumento continuo de la tensión debido a la disminución exponencial del área transversal.

En el modelo mediante elementos finitos, en cambio, la tensión permanece constante dentro de cada elemento. Esto es consecuencia directa de la interpolación lineal del desplazamiento, que produce una deformación constante y, por lo tanto, una tensión constante dentro de cada elemento.

Por esta razón, el campo numérico presenta una forma escalonada. Al utilizar un solo elemento se obtiene un único valor de tensión para toda la barra, lo que no permite representar su aumento hacia el extremo de menor sección. Al utilizar dos y cuatro elementos aparecen, respectivamente, dos y cuatro niveles de tensión, obteniéndose una representación cada vez más cercana al campo continuo.

Además, en la formulación utilizada el área de cada elemento se evalúa en su punto medio. Por lo tanto, el valor de tensión obtenido para cada elemento representa principalmente el comportamiento de la barra en torno a dicha posición. Esto explica que el último elemento del modelo de cuatro elementos entregue una tensión de aproximadamente

$$\sigma_4 = 4.498 \text{ MPa}, \quad (53)$$

mientras que la tensión analítica en el extremo $z = L$ alcanza

$$\sigma(L) = 5.097 \text{ MPa}. \quad (54)$$

Esta diferencia no indica que el modelo esté divergiendo. El último elemento representa todo el intervalo comprendido entre sus dos nodos mediante un único valor constante y no busca reproducir directamente la tensión puntual del extremo. Al continuar refinando la discretización, el punto medio del último elemento se aproxima cada vez más a $z = L$ y, en consecuencia, su tensión también se aproxima al valor analítico en dicho extremo.

Influencia de la discretización

La comparación de las tres discretizaciones permite observar el efecto directo del tamaño de los elementos sobre la precisión de la solución. Al disminuir L_e , la variación real del área dentro de cada elemento también disminuye, por lo que la hipótesis de considerar A_e constante resulta progresivamente más representativa.

De esta forma, el refinamiento de la malla mejora simultáneamente dos aproximaciones presentes en el modelo: la representación lineal del campo de desplazamientos y la representación por tramos constantes de la sección transversal.

Finalmente, la tensión máxima analítica, $\sigma_{\max} = 5.097 \text{ MPa}$, es muy inferior al límite elástico $S_y = 300 \text{ MPa}$, por lo que la hipótesis de comportamiento lineal elástico resulta consistente.

Problema 2: Diseño de estructuras reticuladas

Enunciado

Se propone diseñar un puente a partir de una estructura reticulada plana formada por barras de acero ($E = 206 \text{ GPa}$ y $S_y = 300 \text{ MPa}$) que cubra una longitud determinada ($L = 20 \text{ m}$) y que sea capaz de soportar la acción de una fuerza vertical situada en el centro de la luz ($P = 50 \text{ kN}$).

Las barras a utilizar (el número de barras a considerar en el diseño es de libre elección) tienen sección transversal constante ($A = 1000 \text{ mm}^2$) y una longitud máxima dada ($L_{\max\text{-barra}} = 5 \text{ m}$). Se desprecia en el cálculo mediante MEF el peso propio de las barras.

Como restricción de diseño, se pide que la tensión en ninguna de las barras supere el límite elástico y que la deflexión máxima sea menor a un valor dado ($v_{\max} = 10 \text{ mm}$).

Se piden al menos dos diseños distintos, analizando sus ventajas y desventajas. Adicionalmente, se pide extender a tres dimensiones uno de los diseños anteriores.

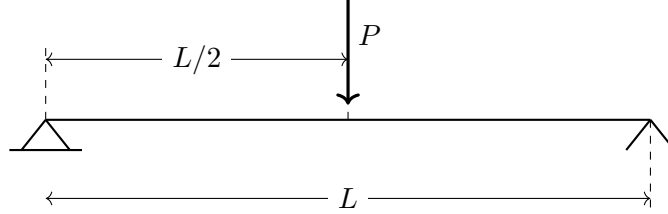


Figura 4. Esquema del problema de diseño de la estructura reticulada.

Formulación mediante elementos finitos en dos dimensiones

A diferencia del problema anterior, en una estructura reticulada bidimensional cada nodo puede desplazarse en dos direcciones. Por lo tanto, cada nodo i posee los grados de libertad

$$\mathbf{u}_i = \begin{bmatrix} u_i \\ v_i \end{bmatrix}, \quad (55)$$

donde u_i corresponde al desplazamiento horizontal y v_i al desplazamiento vertical.

Para una estructura de n_n nodos, el vector global de desplazamientos tiene la forma

$$\mathbf{U} = \begin{bmatrix} u_0 & v_0 & u_1 & v_1 & \cdots & u_{n_n-1} & v_{n_n-1} \end{bmatrix}^T. \quad (56)$$

De esta forma, los grados de libertad asociados a un nodo i se almacenan computacionalmente en las posiciones

$$u_i \rightarrow 2i, \quad v_i \rightarrow 2i + 1. \quad (57)$$

Esta numeración se utiliza posteriormente tanto para ensamblar las matrices elementales como para aplicar cargas y restricciones.

Geometría de los elementos

Cada barra se define a partir de los dos nodos que conecta. Para un elemento e unido a los nodos i y j , sus coordenadas son

$$(x_i, y_i) \quad \text{y} \quad (x_j, y_j). \quad (58)$$

Las diferencias de coordenadas están dadas por

$$\Delta x = x_j - x_i, \quad (59)$$

$$\Delta y = y_j - y_i, \quad (60)$$

y la longitud del elemento corresponde a

$$L_e = \sqrt{(\Delta x)^2 + (\Delta y)^2}. \quad (61)$$

Para representar la orientación del elemento respecto del sistema global se utilizan los cosenos directores

$$c = \frac{\Delta x}{L_e}, \quad s = \frac{\Delta y}{L_e}. \quad (62)$$

En la implementación computacional, estas operaciones se realizan mediante la función `geometria_elementos`, que recorre la matriz de conectividad e identifica los dos nodos pertenecientes a cada barra:

```
nodo_1 = conectividad[i][0]
nodo_2 = conectividad[i][1]
```

A partir de las coordenadas correspondientes se calculan

```
dx = x2 - x1
dy = y2 - y1
```

```
Le = np.sqrt(dx**2 + dy**2)
```

```
cosenos[i] = dx / Le
senos[i] = dy / Le
```

De esta manera, una misma función puede utilizarse para elementos horizontales, verticales o inclinados sin necesidad de definir previamente el ángulo de cada barra.

Matriz de rigidez elemental

En su sistema local, cada barra presenta únicamente deformación axial. Sin embargo, al encontrarse orientada arbitrariamente dentro del plano, su rigidez debe expresarse respecto de los grados de libertad globales

$$\mathbf{u}^{(e)} = \begin{bmatrix} u_i \\ v_i \\ u_j \\ v_j \end{bmatrix}. \quad (63)$$

Utilizando los cosenos directores definidos anteriormente, la matriz de rigidez del elemento en coordenadas globales queda dada por

$$\mathbf{K}^{(e)} = \frac{EA}{L_e} \begin{bmatrix} c^2 & cs & -c^2 & -cs \\ cs & s^2 & -cs & -s^2 \\ -c^2 & -cs & c^2 & cs \\ -cs & -s^2 & cs & s^2 \end{bmatrix}. \quad (64)$$

Esta expresión permite transformar el comportamiento axial de la barra desde su sistema local hacia el sistema global de coordenadas.

En el código, la matriz se construye para cada elemento mediante

```
Ki = (E*A/Le) * np.array([
    [ c**2,  c*s,  -c**2, -c*s],
    [ c*s,   s**2, -c*s,  -s**2],
    [-c**2, -c*s,   c**2,  c*s],
    [-c*s,  -s**2,  c*s,   s**2]
])
```

Por lo tanto, cada barra posee una matriz de rigidez diferente dependiendo de su longitud y orientación, aunque todas poseen las mismas propiedades E y A .

Ensamblaje de la matriz global

La estructura completa se representa mediante una matriz global de dimensiones

$$2n_n \times 2n_n, \quad (65)$$

debido a que cada nodo presenta dos grados de libertad.

Para incorporar cada matriz elemental dentro de la matriz global se deben identificar las posiciones globales asociadas a los dos nodos de la barra. Para un elemento conectado entre los nodos i y j , estas posiciones corresponden a

$$[2i, 2i + 1, 2j, 2j + 1]. \quad (66)$$

En el código esta relación se almacena mediante

```
grados = [
    2*nodo_1,
    2*nodo_1 + 1,
    2*nodo_2,
    2*nodo_2 + 1
]
```

La matriz elemental se incorpora posteriormente en las posiciones correspondientes mediante

```
for j in range(4):
    for k in range(4):
        K[grados[j], grados[k]] += Ki[j, k]
```

El operador += permite acumular las contribuciones de todas las barras que comparten un mismo nodo. De esta forma, la rigidez de cada grado de libertad global queda determinada por la suma de las rigideces de los elementos conectados a él.

Vector de fuerzas

El vector global de fuerzas posee la misma organización que el vector de desplazamientos,

$$\mathbf{F} = \begin{bmatrix} F_{x,0} \\ F_{y,0} \\ F_{x,1} \\ F_{y,1} \\ \vdots \end{bmatrix}. \quad (67)$$

La carga del problema se aplica verticalmente hacia abajo en el nodo correspondiente al centro del vano. Al considerar positiva la dirección vertical hacia arriba, la carga se incorpora con signo negativo.

La función utilizada para generar este vector corresponde a

```
F = np.zeros(2*nodos)
F[2*nodo_carga + 1] = -P
```

El término `2*nodo_carga+1` identifica el grado de libertad vertical del nodo donde se aplica la fuerza.

Condiciones de borde

Para evitar movimientos de cuerpo rígido, la estructura se apoya mediante una articulación en un extremo y un apoyo móvil en el otro.

En el apoyo articulado se restringen los desplazamientos horizontal y vertical,

$$u = 0, \quad v = 0, \quad (68)$$

mientras que en el apoyo móvil únicamente se restringe el movimiento vertical,

$$v = 0. \quad (69)$$

Computacionalmente, los grados de libertad restringidos se almacenan en el vector **restringidos**. A partir de ellos se identifican todos los grados libres mediante

```
libres = []

for i in range(grados_totales):
    if i not in restringidos:
```

```
libres.append(i)
```

Posteriormente se construye el sistema reducido

$$\mathbf{K}_r \mathbf{U}_r = \mathbf{F}_r, \quad (70)$$

utilizando únicamente los grados de libertad no restringidos.

En el código esto se realiza mediante

```
Kr = K[np.ix_(libres, libres)]  
Fr = F[libres]
```

y los desplazamientos desconocidos se calculan a partir de

```
Ur = np.linalg.solve(Kr, Fr)
```

Finalmente, los desplazamientos calculados se incorporan nuevamente al vector global,

```
U = np.zeros(grados_totales)  
U[libres] = Ur
```

de manera que las posiciones correspondientes a los apoyos mantienen automáticamente un desplazamiento nulo.

Cálculo de deformaciones y tensiones

Una vez obtenido el vector global de desplazamientos, la deformación axial de cada barra se calcula proyectando el desplazamiento relativo entre sus nodos sobre la dirección longitudinal del elemento.

Para una barra conectada entre los nodos i y j ,

$$\varepsilon_e = \frac{c(u_j - u_i) + s(v_j - v_i)}{L_e}. \quad (71)$$

La expresión anterior representa el cambio de longitud de la barra dividido por su longitud original. Los términos asociados a c y s permiten considerar únicamente la componente del movimiento relativo que ocurre en la dirección del eje del elemento.

En la implementación se utiliza

```
deformaciones[i] = (  
    c*(u2-u1) + s*(v2-v1)  
) / Le
```

Una deformación positiva corresponde a un alargamiento de la barra, mientras que una deformación negativa indica un acortamiento.

Finalmente, considerando nuevamente un comportamiento lineal elástico, la tensión axial se calcula mediante

$$\sigma_e = E\varepsilon_e. \quad (72)$$

De esta forma,

$$\sigma_e > 0 \quad (73)$$

representa una barra sometida a tracción, mientras que

$$\sigma_e < 0 \quad (74)$$

corresponde a una barra sometida a compresión.

La implementación computacional utiliza la misma función empleada anteriormente,

`tensiones = E * deformaciones`

ya que la relación constitutiva del material no cambia al pasar de una barra unidimensional a una estructura reticulada bidimensional.

Diseño 1

Geometría y conectividad

Como primera alternativa se propone una estructura reticulada formada principalmente por una sucesión de elementos triangulares. La geometría seleccionada cubre completamente el vano de 20 m y posee una altura de 4 m.

La estructura está compuesta por 9 nodos y 15 barras. Los nodos inferiores se encuentran separados 5 m entre sí, mientras que los nodos superiores se sitúan en los puntos medios de cada tramo inferior, tal como se muestra en la Figura 5.

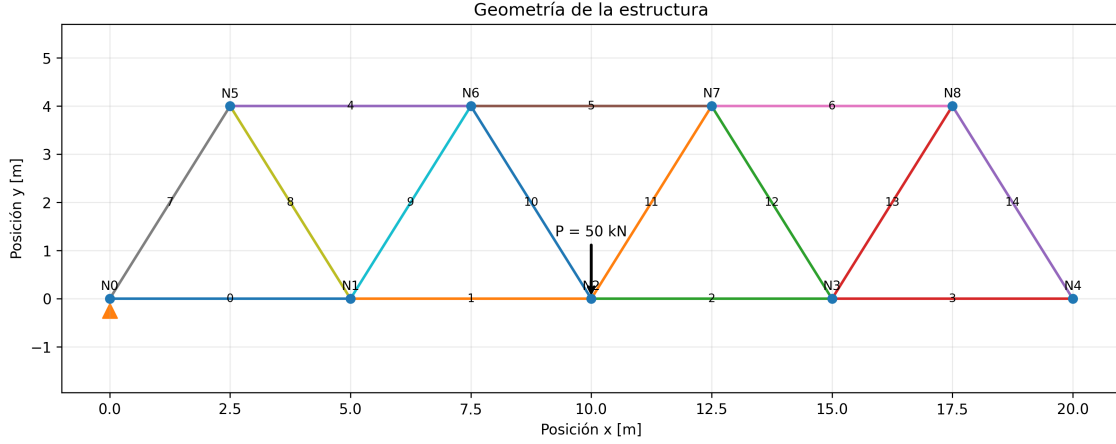


Figura 5. Geometría, numeración de nodos y barras del Diseño 1.

La geometría del modelo se definió a partir de las coordenadas nodales mostradas en la Figura 5, almacenadas computacionalmente en la variable `coordenadas`.

La conectividad define qué pares de nodos forman cada elemento. Para este diseño se utilizaron cuatro barras inferiores, tres barras superiores y ocho barras diagonales. A partir de estas coordenadas, la función `geometria_elementos` calcula automáticamente la longitud y orientación de cada barra.

Las barras inferiores poseen una longitud de

$$L_h = 5.00 \text{ m}, \quad (75)$$

mientras que las diagonales poseen una longitud

$$L_d = \sqrt{(2.5)^2 + (4)^2} = 4.72 \text{ m}. \quad (76)$$

Por lo tanto, la longitud máxima presente en la estructura es

$$\boxed{L_{\max} = 5.00 \text{ m}}, \quad (77)$$

cumpléndose la restricción geométrica establecida en el enunciado.

Condiciones de borde y aplicación de la carga

En el extremo izquierdo, correspondiente al nodo 0, se considera un apoyo articulado. Por lo tanto, se restringen tanto el desplazamiento horizontal como el vertical,

$$u_0 = 0, \quad v_0 = 0. \quad (78)$$

En el extremo derecho, correspondiente al nodo 4, se utiliza un apoyo móvil, restringiéndose únicamente el desplazamiento vertical,

$$v_4 = 0. \quad (79)$$

Debido a la numeración global de grados de libertad utilizada,

$$u_i \rightarrow 2i, \quad v_i \rightarrow 2i + 1, \quad (80)$$

las restricciones se implementan en el código mediante

```
restringidos = [0, 1, 9]
```

La fuerza vertical de 50 kN se aplica en el centro del vano, correspondiente al nodo 2, ubicado en $x = 10$ m. El vector global de fuerzas se genera mediante

```
F = vector_fuerzas(nodos, 2, P)
```

de modo que la componente vertical de dicho nodo toma el valor

$$F_{y,2} = -50000 \text{ N}. \quad (81)$$

Una vez incorporadas las restricciones, se resuelve el sistema reducido y se obtienen los desplazamientos nodales mediante la función `resolver_sistema`.

Resultados

A partir de los desplazamientos nodales se calcularon las deformaciones y tensiones axiales de cada barra. La distribución obtenida se presenta en la Figura 6.

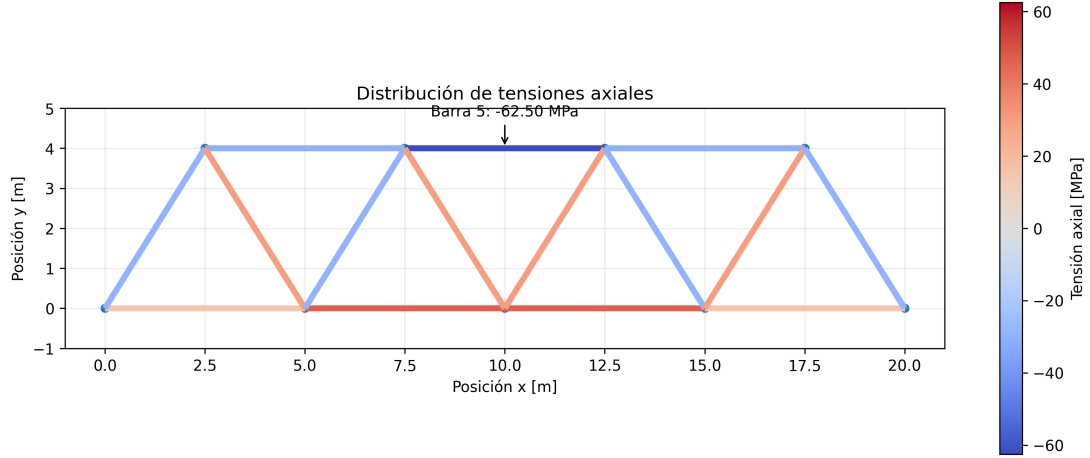


Figura 6. Distribución de tensiones axiales correspondiente al Diseño 1. Los valores positivos representan tracción y los negativos compresión.

La estructura presenta barras sometidas tanto a tracción como a compresión. La máxima magnitud de tensión se obtiene en la barra 5, perteneciente al cordón superior central, con un valor

$$\sigma_{\max, \text{abs}} = 62.50 \text{ MPa} . \quad (82)$$

El signo de la tensión en dicha barra es negativo,

$$\sigma_5 = -62.50 \text{ MPa}, \quad (83)$$

por lo que corresponde a un elemento sometido a compresión.

Por otra parte, a partir de las componentes verticales del vector global de desplazamientos,

`desplazamientos_verticales = U[1::2]`

se obtiene una deflexión vertical máxima de

$$|v|_{\max} = 8.399 \text{ mm} . \quad (84)$$

Esta deflexión ocurre en el nodo central inferior, donde se encuentra aplicada la carga vertical.

Debido a la simetría geométrica y de carga, las reacciones verticales obtenidas en ambos apoyos son iguales,

$$R_{y,0} = R_{y,4} = 25 \text{ kN}, \quad (85)$$

verificándose además el equilibrio vertical global,

$$R_{y,0} + R_{y,4} = 25 + 25 = 50 \text{ kN} = P. \quad (86)$$

Verificación de las restricciones de diseño

Los principales resultados obtenidos para el Diseño 1 se resumen en la Tabla 3.

Tabla 3: Verificación de las restricciones de diseño para el Diseño 1.

Parámetro	Resultado	Límite	Cumplimiento
Número de barras	15	—	—
Longitud máxima	5.00 m	5.00 m	Sí
Deflexión máxima	8.399 mm	10 mm	Sí
Tensión máxima absoluta	62.50 MPa	300 MPa	Sí

La tensión máxima se encuentra ampliamente por debajo del límite elástico del material, mientras que la deflexión alcanza aproximadamente un 84% del valor admisible. Por lo tanto, el Diseño 1 satisface las restricciones de longitud máxima, tensión y desplazamiento establecidas.

Diseño 2

Geometría y conectividad

Como segunda alternativa se propone una estructura reticulada con montantes verticales y diagonales orientadas hacia la zona central del vano. A diferencia del Diseño 1, se utiliza una mayor cantidad de elementos y una separación horizontal entre nodos de 2.5 m.

La estructura posee una longitud total de 20 m, una altura de 4.2 m, 18 nodos y 33 barras. La configuración adoptada se presenta en la Figura 7.

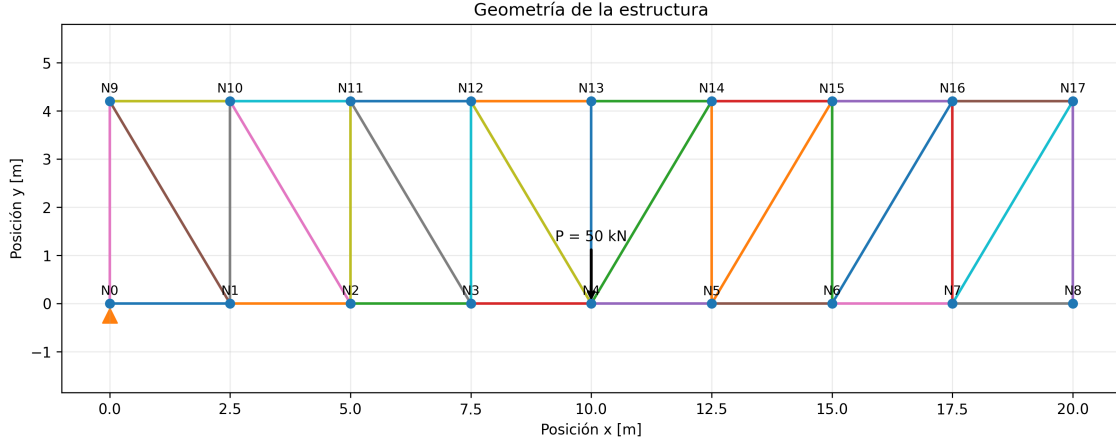


Figura 7. Geometría y numeración de nodos correspondiente al Diseño 2.

Los nodos inferiores se distribuyen entre $x = 0$ m y $x = 20$ m, separados 2.5 m entre sí. Sobre cada uno de ellos se dispone un nodo superior a una altura de 4.2 m.

La conectividad considera ocho barras en el cordón inferior, ocho en el cordón superior, nueve barras verticales y ocho diagonales, resultando en un total de

$$n_{\text{barras}} = 33. \quad (87)$$

Las barras horizontales poseen una longitud de 2.5 m, mientras que los montantes verticales poseen una longitud de 4.2 m. Las diagonales presentan una longitud

$$L_d = \sqrt{(2.5)^2 + (4.2)^2} = 4.89 \text{ m}. \quad (88)$$

Por lo tanto,

$$\boxed{L_{\text{max}} = 4.89 \text{ m}} \quad (89)$$

y se cumple la restricción de longitud máxima de 5 m.

Condiciones de borde y carga

Se mantienen las mismas condiciones de apoyo utilizadas en el Diseño 1. El nodo 0 se considera articulado, mientras que el nodo 8 corresponde al apoyo móvil.

En consecuencia, los grados de libertad restringidos utilizados en este caso son

`restringidos_2 = [0, 1, 17]`

La carga de 50 kN se aplica verticalmente en el nodo 4, correspondiente al centro del vano,

`F_2 = vector_fuerzas(nodos_2, 4, P)`

manteniéndose así las mismas condiciones externas utilizadas para evaluar el Diseño 1.

Resultados

La distribución de tensiones axiales obtenida mediante el modelo de elementos finitos se presenta en la Figura 8.

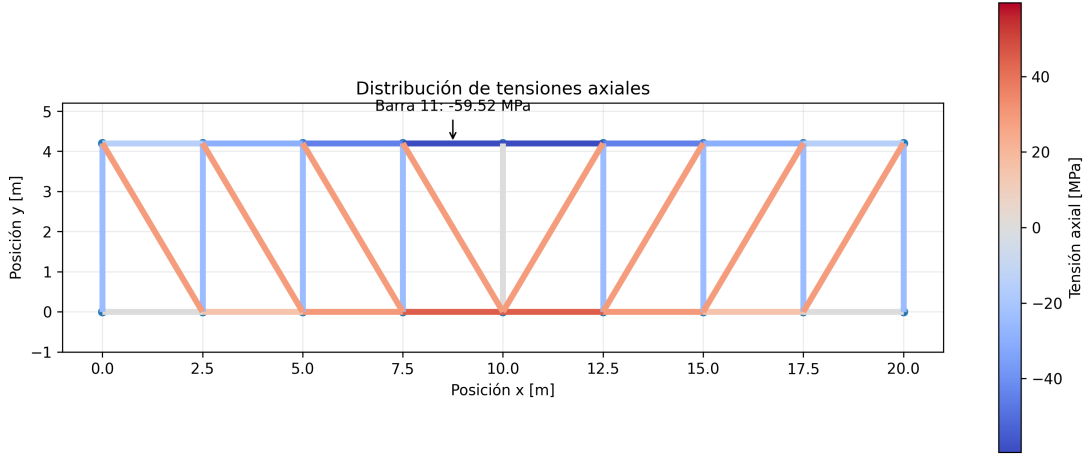


Figura 8. Distribución de tensiones axiales correspondiente al Diseño 2. Los valores positivos representan tracción y los negativos compresión.

La distribución presenta nuevamente elementos sometidos tanto a tracción como a compresión. La mayor magnitud de tensión se obtiene en la barra 11, perteneciente al cordón superior, con un valor

$$\sigma_{11} = -59.52 \text{ MPa} . \quad (90)$$

Por lo tanto, la tensión máxima en valor absoluto corresponde a

$$|\sigma|_{\max} = 59.52 \text{ MPa} . \quad (91)$$

El signo negativo indica que la barra crítica se encuentra sometida a compresión.

Por otra parte, el análisis de los desplazamientos nodales entrega una deflexión vertical máxima de

$$\boxed{|v|_{\max} = 9.982 \text{ mm}}. \quad (92)$$

El máximo desplazamiento vertical se produce en la zona central del vano, donde se encuentra aplicada la carga.

Debido a la simetría de la estructura y de la carga aplicada, las reacciones verticales en los extremos corresponden nuevamente a

$$R_{y,0} = R_{y,8} = 25 \text{ kN}, \quad (93)$$

satisfaciendo el equilibrio global de fuerzas verticales.

Verificación de las restricciones de diseño

Los resultados principales del Diseño 2 se presentan en la Tabla 4.

Tabla 4: Verificación de las restricciones de diseño para el Diseño 2.

Parámetro	Resultado	Límite	Cumplimiento
Número de barras	33	–	–
Longitud máxima	4.89 m	5.00 m	Sí
Deflexión máxima	9.982 mm	10 mm	Sí
Tensión máxima absoluta	59.52 MPa	300 MPa	Sí

La tensión máxima se mantiene ampliamente por debajo del límite elástico. En cambio, la deflexión alcanza aproximadamente un 99.8% del valor admisible, por lo que el diseño cumple esta restricción con un margen reducido.

En consecuencia, el Diseño 2 cumple las tres restricciones impuestas: longitud máxima de barra, límite elástico y deflexión máxima.

Comparación de los diseños bidimensionales

Los principales resultados obtenidos para ambas configuraciones se resumen en la Tabla 5.

Tabla 5: Comparación entre los diseños bidimensionales propuestos.

Parámetro	Diseño 1	Diseño 2
Número de nodos	9	18
Número de barras	15	33
Longitud máxima [m]	5.00	4.89
Tensión máxima absoluta [MPa]	62.50	59.52
Deflexión máxima [mm]	8.399	9.982

Ambos diseños cumplen las restricciones impuestas. El Diseño 2 presenta una tensión máxima ligeramente menor, pero requiere más del doble de barras y alcanza una deflexión muy cercana al límite permitido. El Diseño 1, en cambio, presenta una menor deflexión con una configuración considerablemente más simple. Por este motivo se selecciona el Diseño 1 para realizar la extensión tridimensional.

Extensión tridimensional del Diseño 1

A partir del Diseño 1 se construyó una estructura tridimensional duplicando la reticulada original en una dirección perpendicular a su plano. Las dos caras se separaron una distancia de 1.5 m y se conectaron mediante barras transversales y diagonales, con el objetivo de otorgar rigidez a la estructura fuera del plano original.

De esta manera, la estructura tridimensional está compuesta por dos reticulados equivalentes al Diseño 1 unidos entre sí. La geometría resultante y la distribución de tensiones axiales obtenida se presentan en la Figura 9.

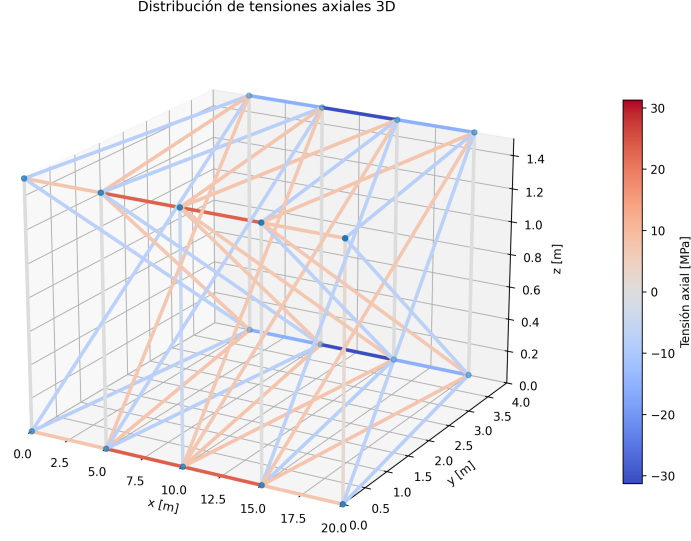


Figura 9. Geometría tridimensional y distribución de tensiones axiales de la extensión del Diseño 1.

Formulación del elemento tridimensional

La principal diferencia respecto del modelo bidimensional corresponde al número de grados de libertad por nodo. En el caso tridimensional, cada nodo puede desplazarse en las tres direcciones espaciales,

$$\mathbf{u}_i = \begin{bmatrix} u_i \\ v_i \\ w_i \end{bmatrix}. \quad (94)$$

Por lo tanto, los grados de libertad asociados al nodo i se identifican como

$$u_i \rightarrow 3i, \quad v_i \rightarrow 3i + 1, \quad w_i \rightarrow 3i + 2. \quad (95)$$

La longitud de cada barra se calcula ahora considerando las tres componentes espaciales,

$$L_e = \sqrt{(\Delta x)^2 + (\Delta y)^2 + (\Delta z)^2}. \quad (96)$$

Asimismo, se utilizan tres cosenos directores,

$$l = \frac{\Delta x}{L_e}, \quad m = \frac{\Delta y}{L_e}, \quad n = \frac{\Delta z}{L_e}. \quad (97)$$

Como consecuencia, la matriz de rigidez de cada barra pasa de una dimensión 4×4 a una matriz 6×6 ,

$$\mathbf{K}^{(e)} = \frac{EA}{L_e} \begin{bmatrix} l^2 & lm & ln & -l^2 & -lm & -ln \\ lm & m^2 & mn & -lm & -m^2 & -mn \\ ln & mn & n^2 & -ln & -mn & -n^2 \\ -l^2 & -lm & -ln & l^2 & lm & ln \\ -lm & -m^2 & -mn & lm & m^2 & mn \\ -ln & -mn & -n^2 & ln & mn & n^2 \end{bmatrix}. \quad (98)$$

La lógica de ensamblaje y resolución del sistema se mantiene respecto del caso bidimensional, modificándose únicamente el número de grados de libertad asociados a cada nodo.

Aplicación de la carga y condiciones de borde

La carga total de 50 kN se distribuyó simétricamente entre los dos nodos centrales correspondientes a las dos caras de la estructura,

$$P_1 = P_2 = 25 \text{ kN}, \quad (99)$$

de manera que

$$P_1 + P_2 = 50 \text{ kN}. \quad (100)$$

Las restricciones de los apoyos fueron adaptadas al modelo tridimensional para impedir movimientos de cuerpo rígido tanto en el plano original como en la dirección transversal.

Resultados y verificación

La incorporación de una segunda reticulada y de elementos transversales permite repartir la carga entre ambas caras de la estructura. El análisis mediante MEF entrega una tensión máxima absoluta aproximada de

$$|\sigma|_{\max} \approx 31.25 \text{ MPa}. \quad (101)$$

La deflexión vertical máxima obtenida es aproximadamente

$$|v|_{\max} \approx 3.46 \text{ mm}. \quad (102)$$

La longitud máxima de las barras utilizadas se mantiene dentro de la restricción establecida,

$$L_{\max} = 5.00 \text{ m}. \quad (103)$$

Los resultados se resumen en la Tabla 6.

Tabla 6: Verificación de las restricciones para la extensión tridimensional.

Parámetro	Resultado	Límite	Cumplimiento
Longitud máxima	5.00 m	5.00 m	Sí
Deflexión máxima	3.46 mm	10 mm	Sí
Tensión máxima absoluta	31.25 MPa	300 MPa	Sí

La extensión tridimensional cumple las tres restricciones establecidas y presenta menores tensiones y desplazamientos que el Diseño 1 bidimensional. Esta mejora se obtiene a costa de incorporar una segunda reticulada y elementos adicionales de arriostramiento.

Análisis y discusión

Los resultados muestran que aumentar el número de barras no implica necesariamente aumentar la rigidez global de una estructura. Aunque el Diseño 2 utiliza más del doble de elementos que el Diseño 1, presenta una mayor deflexión. Esto evidencia la importancia de la geometría y de la orientación de las barras en la transmisión de las cargas hacia los apoyos.

En ambos diseños las tensiones se mantienen ampliamente por debajo del límite elástico, por lo que la restricción más exigente corresponde al desplazamiento. En este aspecto, el Diseño 1 presenta un mayor margen frente al valor admisible y además requiere una configuración considerablemente más simple. Por estas razones se considera la alternativa bidimensional más conveniente entre las estudiadas.

La extensión tridimensional mejora significativamente la rigidez al incorporar una segunda reticulada y nuevos caminos de transmisión de carga. La disminución de tensiones y desplazamientos no se debe únicamente a trabajar en tres dimensiones, sino también al aumento de

material y al arriostramiento entre ambas caras. Por lo tanto, esta mejora estructural se obtiene a costa de una mayor cantidad de elementos y una mayor complejidad constructiva.

Código computacional

El código utilizado para el desarrollo de ambos problemas fue implementado en Python. A continuación se presentan las funciones y bloques empleados para la solución analítica, la formulación mediante elementos finitos y la generación de los resultados mostrados en el informe.

Problema 1: barra de sección variable

Librerías y solución analítica

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3  import pandas as pd
4  import os
5  from matplotlib.collections import LineCollection
6  from matplotlib.colors import Normalize
7  from mpl_toolkits.mplot3d.art3d import Line3DCollection
8
9
10 def solucion_analitica(z, P, a_0, E, L):
11
12     tension = (P / a_0) * np.exp(z / L)
13
14     desplazamiento = (P * L / (E * a_0)) * (
15         np.exp(z / L) - 1
16     )
17
18     return desplazamiento, tension
19
20
21 a_0 = 1600
22 L = 10000
23 E = 206000
24 P = 3000
25
26 z = np.linspace(0, L, 100)
27
28 w_analitico, sigma_analitico = solucion_analitica(
29     z, P, a_0, E, L
30 )
```

Funciones para la solución mediante MEF

```
1  def centros_nodos(nodos, L):
2
3      z_nodos = np.linspace(0, L, nodos)
```

```

4
5     zc_nodos = np.zeros(nodos)
6
7     for i in range(len(zc_nodos)):
8
9         if i == 0:
10
11             zc_nodos[i] = 0
12
13         else:
14
15             zc_nodos[i] = (
16                 (z_nodos[i] - z_nodos[i-1]) / 2
17                 + z_nodos[i-1]
18             )
19
20     return zc_nodos
21
22
23 def calcular_vector_k(elementos, L, E, area_nodo):
24
25     vector_k = np.zeros(elementos)
26
27     for i in range(len(vector_k)):
28
29         vector_k[i] = (
30             E * area_nodo[i + 1] / (L/elementos)
31         )
32
33     return vector_k
34
35
36 def matriz_k(vector_k, nodos):
37
38     K = np.zeros((nodos, nodos))
39
40     for i in range(nodos - 1):
41
42         Ki = vector_k[i] * np.array([
43             [1, -1],
44             [-1, 1]
45         ])
46
47         K[i:i+2, i:i+2] += Ki
48
49     return K
50
51
52 def resolver_sistema(K, nodos, P):
53

```

```

54     F = np.zeros(nodos)
55
56     F[-1] = P
57
58     Kr = K[1:, 1:]
59     Fr = F[1:]
60
61     Ur = np.linalg.solve(Kr, Fr)
62
63     U = np.zeros(nodos)
64
65     U[1:] = Ur
66
67     return U
68
69
70 def calcular_deformaciones(U, elementos, L):
71
72     Le = L / elementos
73
74     deformaciones = np.zeros(elementos)
75
76     for i in range(elementos):
77
78         deformaciones[i] = (
79             U[i + 1] - U[i]
80         ) / Le
81
82     return deformaciones
83
84
85 def calcular_tensiones(deformaciones, E):
86
87     tensiones = E * deformaciones
88
89     return tensiones

```

Discretización de un elemento

```

1  elementos = 1
2  nodos = elementos + 1
3
4  a_0 = 1600
5  L = 10000
6  E = 206000
7  P = 3000
8
9  zc_nodos = centros_nodos(nodos, L)
10

```

```

11 area_nodo = a_0 * np.exp(-zc_nodos / L)
12
13 vector_k = calcular_vector_k(
14     elementos, L, E, area_nodo
15 )
16
17 K = matriz_k(vector_k, nodos)
18
19 resolucion_1 = resolver_sistema(
20     K, nodos, P
21 )
22
23 deformaciones_1 = calcular_deformaciones(
24     resolucion_1, elementos, L
25 )
26
27 tensiones_1 = calcular_tensiones(
28     deformaciones_1, E
29 )

```

Discretización de dos elementos

```

1 elementos = 2
2 nodos = elementos + 1
3
4 a_0 = 1600
5 L = 10000
6 E = 206000
7 P = 3000
8
9 zc_nodos = centros_nodos(nodos, L)
10
11 area_nodo = a_0 * np.exp(-zc_nodos / L)
12
13 vector_k = calcular_vector_k(
14     elementos, L, E, area_nodo
15 )
16
17 K = matriz_k(vector_k, nodos)
18
19 resolucion_2 = resolver_sistema(
20     K, nodos, P
21 )
22
23 deformaciones_2 = calcular_deformaciones(
24     resolucion_2, elementos, L
25 )
26
27 tensiones_2 = calcular_tensiones(

```



```

28     deformaciones_2, E
29 )

```

Discretización de cuatro elementos

```

1  elementos = 4
2  nodos = elementos + 1
3
4  a_0 = 1600
5  L = 10000
6  E = 206000
7  P = 3000
8
9  zc_nodos = centros_nodos(nodos, L)
10
11 area_nodo = a_0 * np.exp(-zc_nodos / L)
12
13 vector_k = calcular_vector_k(
14     elementos, L, E, area_nodo
15 )
16
17 K = matriz_k(vector_k, nodos)
18
19 resolucion_4 = resolver_sistema(
20     K, nodos, P
21 )
22
23 deformaciones_4 = calcular_deformaciones(
24     resolucion_4, elementos, L
25 )
26
27 tensiones_4 = calcular_tensiones(
28     deformaciones_4, E
29 )

```

Comparación de resultados

```

1  z_nodos_1 = np.linspace(0, L, 2)
2  z_nodos_2 = np.linspace(0, L, 3)
3  z_nodos_4 = np.linspace(0, L, 5)
4
5  plt.figure(figsize=(8, 5))
6
7  plt.plot(
8      z,
9      w_analitico,
10     label="Solucion analitica"

```

```

11 )
12
13 plt.plot(
14     z_nodos_1,
15     resolucion_1,
16     marker="o",
17     label="MEF - 1 elemento"
18 )
19
20 plt.plot(
21     z_nodos_2,
22     resolucion_2,
23     marker="o",
24     label="MEF - 2 elementos"
25 )
26
27 plt.plot(
28     z_nodos_4,
29     resolucion_4,
30     marker="o",
31     label="MEF - 4 elementos"
32 )
33
34 plt.xlabel("Posicion z [mm]")
35 plt.ylabel("Desplazamiento w [mm]")
36 plt.title("Campo de desplazamientos")
37 plt.grid()
38 plt.legend()
39 plt.tight_layout()
40
41 plt.show()
42
43
44 plt.figure(figsize=(8, 5))
45
46 plt.plot(
47     z,
48     sigma_analitico,
49     label="Solucion analitica"
50 )
51
52 plt.stairs(
53     tensiones_1,
54     z_nodos_1,
55     label="MEF - 1 elemento"
56 )
57
58 plt.stairs(
59     tensiones_2,
60     z_nodos_2,

```

```

61     label="MEF - 2 elementos"
62 )
63
64 plt.stairs(
65     tensiones_4,
66     z_nodos_4,
67     label="MEF - 4 elementos"
68 )
69
70 plt.xlabel("Posicion z [mm]")
71 plt.ylabel("Tension axial [MPa]")
72 plt.title("Campo de tensiones")
73 plt.grid()
74 plt.legend()
75 plt.tight_layout()
76
77 plt.show()
78
79
80 w_exacto = w_analitico[-1]
81
82 error_1 = abs(
83     (resolucion_1[-1] - w_exacto) / w_exacto
84 ) * 100
85
86 error_2 = abs(
87     (resolucion_2[-1] - w_exacto) / w_exacto
88 ) * 100
89
90 error_4 = abs(
91     (resolucion_4[-1] - w_exacto) / w_exacto
92 ) * 100
93
94 tabla_error = pd.DataFrame({
95     "Elementos": [1, 2, 4],
96
97     "Desplazamiento MEF [mm]": [
98         resolucion_1[-1],
99         resolucion_2[-1],
100         resolucion_4[-1]
101     ],
102
103     "Desplazamiento analitico [mm]": [
104         w_exacto,
105         w_exacto,
106         w_exacto
107     ],
108
109     "Error [%]": [
110         error_1,

```

```

111         error_2,
112         error_4
113     ]
114 })

```

Problema 2: estructuras reticuladas bidimensionales

Funciones generales para elementos de barra 2D

```

1  def geometria_elementos(coordenadas, conectividad):
2
3      elementos = len(conectividad)
4
5      longitudes = np.zeros(elementos)
6      cosenos = np.zeros(elementos)
7      senos = np.zeros(elementos)
8
9      for i in range(elementos):
10
11         nodo_1 = conectividad[i][0]
12         nodo_2 = conectividad[i][1]
13
14         x1 = coordenadas[nodo_1][0]
15         y1 = coordenadas[nodo_1][1]
16
17         x2 = coordenadas[nodo_2][0]
18         y2 = coordenadas[nodo_2][1]
19
20         dx = x2 - x1
21         dy = y2 - y1
22
23         Le = np.sqrt(dx**2 + dy**2)
24
25         longitudes[i] = Le
26         cosenos[i] = dx / Le
27         senos[i] = dy / Le
28
29     return longitudes, cosenos, senos
30
31
32 def matriz_k(coordenadas, conectividad, E, A):
33
34     nodos = len(coordenadas)
35     elementos = len(conectividad)
36
37     K = np.zeros((2*nodos, 2*nodos))
38
39     longitudes, cosenos, senos = geometria_elementos(
40         coordenadas,

```

```

41         conectividad
42     )
43
44     for i in range(elementos):
45
46         Le = longitudes[i]
47         c = cosenos[i]
48         s = senos[i]
49
50         Ki = (E * A / Le) * np.array([
51             [ c**2, c*s, -c**2, -c*s],
52             [ c*s, s**2, -c*s, -s**2],
53             [-c**2, -c*s, c**2, c*s],
54             [-c*s, -s**2, c*s, s**2]
55         ])
56
57         nodo_1 = conectividad[i][0]
58         nodo_2 = conectividad[i][1]
59
60         grados = [
61             2*nodo_1,
62             2*nodo_1 + 1,
63             2*nodo_2,
64             2*nodo_2 + 1
65         ]
66
67         for j in range(4):
68
69             for k in range(4):
70
71                 K[
72                     grados[j],
73                     grados[k]
74                 ] += Ki[j, k]
75
76         return K
77
78
79 def resolver_sistema(K, F, restringidos):
80
81     grados_totales = len(F)
82
83     libres = []
84
85     for i in range(grados_totales):
86
87         if i not in restringidos:
88
89             libres.append(i)
90

```

```

91     Kr = K[np.ix_(libres, libres)]
92
93     Fr = F[libres]
94
95     Ur = np.linalg.solve(Kr, Fr)
96
97     U = np.zeros(grados_totales)
98
99     U[libres] = Ur
100
101     return U
102
103
104 def calcular_deformaciones(
105     U,
106     coordenadas,
107     conectividad
108 ):
109
110     elementos = len(conectividad)
111
112     deformaciones = np.zeros(elementos)
113
114     longitudes, cosenos, senos = geometria_elementos(
115         coordenadas,
116         conectividad
117     )
118
119     for i in range(elementos):
120
121         nodo_1 = conectividad[i][0]
122         nodo_2 = conectividad[i][1]
123
124         Le = longitudes[i]
125
126         c = cosenos[i]
127         s = senos[i]
128
129         u1 = U[2*nodo_1]
130         v1 = U[2*nodo_1 + 1]
131
132         u2 = U[2*nodo_2]
133         v2 = U[2*nodo_2 + 1]
134
135         deformaciones[i] = (
136             c*(u2-u1)
137             + s*(v2-v1)
138         ) / Le
139
140     return deformaciones

```

```

141
142
143 def vector_fuerzas(nodos, nodo_carga, P):
144
145     F = np.zeros(2*nodos)
146
147     F[2*nodo_carga + 1] = -P
148
149     return F
150
151
152 def calcular_tensiones(deformaciones, E):
153
154     tensiones = E * deformaciones
155
156     return tensiones

```

Diseño 1

```

1  E = 206000
2  A = 1000
3  P = 50000
4
5  coordenadas = np.array([
6      [0, 0],
7      [5000, 0],
8      [10000, 0],
9      [15000, 0],
10     [20000, 0],
11     [2500, 4000],
12     [7500, 4000],
13     [12500, 4000],
14     [17500, 4000]
15 ])
16
17 nodos = len(coordenadas)
18
19 conectividad = np.array([
20     [0, 1],
21     [1, 2],
22     [2, 3],
23     [3, 4],
24
25     [5, 6],
26     [6, 7],
27     [7, 8],
28
29     [0, 5],
30     [5, 1],

```

```

31
32     [1, 6],
33     [6, 2],
34
35     [2, 7],
36     [7, 3],
37
38     [3, 8],
39     [8, 4]
40 ])
41
42 longitudes, cosenos, senos = geometria_elementos(
43     coordenadas,
44     conectividad
45 )
46
47 K = matriz_k(
48     coordenadas,
49     conectividad,
50     E,
51     A
52 )
53
54 F = vector_fuerzas(
55     nodos,
56     2,
57     P
58 )
59
60 restringidos = [0, 1, 9]
61
62 U = resolver_sistema(
63     K,
64     F,
65     restringidos
66 )
67
68 deformaciones = calcular_deformaciones(
69     U,
70     coordenadas,
71     conectividad
72 )
73
74 tensiones = calcular_tensiones(
75     deformaciones,
76     E
77 )
78
79 reacciones = K @ U - F
80

```



```

81 desplazamientos_verticales = U[1::2]
82
83 desplazamiento_maximo = np.max(
84     np.abs(desplazamientos_verticales)
85 )
86
87 tension_maxima = np.max(
88     np.abs(tensiones)
89 )
90
91 longitud_maxima = np.max(
92     longitudes
93 )

```

Diseño 2

```

1  E = 206000
2  A = 1000
3  P = 50000
4
5  coordenadas_2 = np.array([
6      [0, 0],
7      [2500, 0],
8      [5000, 0],
9      [7500, 0],
10     [10000, 0],
11     [12500, 0],
12     [15000, 0],
13     [17500, 0],
14     [20000, 0],
15     [0, 4200],
16     [2500, 4200],
17     [5000, 4200],
18     [7500, 4200],
19     [10000, 4200],
20     [12500, 4200],
21     [15000, 4200],
22     [17500, 4200],
23     [20000, 4200]
24 ])
25
26 conectividad_2 = np.array([
27     [0, 1],
28     [1, 2],
29     [2, 3],
30     [3, 4],
31     [4, 5],
32     [5, 6],
33     [6, 7],

```

```

34     [7, 8],
35
36     [9, 10],
37     [10, 11],
38     [11, 12],
39     [12, 13],
40     [13, 14],
41     [14, 15],
42     [15, 16],
43     [16, 17],
44
45     [0, 9],
46     [1, 10],
47     [2, 11],
48     [3, 12],
49     [4, 13],
50     [5, 14],
51     [6, 15],
52     [7, 16],
53     [8, 17],
54
55     [9, 1],
56     [10, 2],
57     [11, 3],
58     [12, 4],
59
60     [17, 7],
61     [16, 6],
62     [15, 5],
63     [14, 4]
64 ])
65
66 nodos_2 = len(coordenadas_2)
67
68 longitudes_2, cosenos_2, senos_2 = (
69     geometria_elementos(
70         coordenadas_2,
71         conectividad_2
72     )
73 )
74
75 K_2 = matriz_k(
76     coordenadas_2,
77     conectividad_2,
78     E,
79     A
80 )
81
82 F_2 = vector_fuerzas(
83     nodos_2,

```

```

84     4,
85     P
86 )
87
88 restringidos_2 = [0, 1, 17]
89
90 U_2 = resolver_sistema(
91     K_2,
92     F_2,
93     restringidos_2
94 )
95
96 deformaciones_2 = calcular_deformaciones(
97     U_2,
98     coordenadas_2,
99     conectividad_2
100 )
101
102 tensiones_2 = calcular_tensiones(
103     deformaciones_2,
104     E
105 )
106
107 reacciones_2 = K_2 @ U_2 - F_2
108
109 desplazamientos_verticales_2 = U_2[1::2]
110
111 desplazamiento_maximo_2 = np.max(
112     np.abs(desplazamientos_verticales_2)
113 )
114
115 tension_maxima_2 = np.max(
116     np.abs(tensiones_2)
117 )
118
119 longitud_maxima_2 = np.max(
120     longitudes_2
121 )

```

Extensión tridimensional del Diseño 1

Funciones para elementos de barra 3D

```

1 def geometria_elementos_3D(
2     coordenadas,
3     conectividad
4 ):
5
6     elementos = len(conectividad)

```

```

7
8     longitudes = np.zeros(elementos)
9
10    l = np.zeros(elementos)
11    m = np.zeros(elementos)
12    n = np.zeros(elementos)
13
14    for i in range(elementos):
15
16        nodo_1 = conectividad[i][0]
17        nodo_2 = conectividad[i][1]
18
19        x1 = coordenadas[nodo_1][0]
20        y1 = coordenadas[nodo_1][1]
21        z1 = coordenadas[nodo_1][2]
22
23        x2 = coordenadas[nodo_2][0]
24        y2 = coordenadas[nodo_2][1]
25        z2 = coordenadas[nodo_2][2]
26
27        dx = x2 - x1
28        dy = y2 - y1
29        dz = z2 - z1
30
31        Le = np.sqrt(
32            dx**2
33            + dy**2
34            + dz**2
35        )
36
37        longitudes[i] = Le
38
39        l[i] = dx / Le
40        m[i] = dy / Le
41        n[i] = dz / Le
42
43    return longitudes, l, m, n
44
45
46    def matriz_k_3D(
47        coordenadas,
48        conectividad,
49        E,
50        A
51    ):
52
53        nodos = len(coordenadas)
54
55        elementos = len(conectividad)
56

```

```

57 K = np.zeros(
58     (3*nodos, 3*nodos)
59 )
60
61 longitudes, l, m, n = (
62     geometria_elementos_3D(
63         coordenadas,
64         conectividad
65     )
66 )
67
68 for i in range(elementos):
69
70     Le = longitudes[i]
71
72     lx = l[i]
73     ly = m[i]
74     lz = n[i]
75
76     Ki = (E * A / Le) * np.array([
77         [ lx**2, lx*ly, lx*lz,
78           -lx**2, -lx*ly, -lx*lz],
79
80         [ lx*ly, ly**2, ly*lz,
81           -lx*ly, -ly**2, -ly*lz],
82
83         [ lx*lz, ly*lz, lz**2,
84           -lx*lz, -ly*lz, -lz**2],
85
86         [-lx**2, -lx*ly, -lx*lz,
87           lx**2, lx*ly, lx*lz],
88
89         [-lx*ly, -ly**2, -ly*lz,
90           lx*ly, ly**2, ly*lz],
91
92         [-lx*lz, -ly*lz, -lz**2,
93           lx*lz, ly*lz, lz**2]
94     ])
95
96     nodo_1 = conectividad[i][0]
97     nodo_2 = conectividad[i][1]
98
99     grados = [
100         3*nodo_1,
101         3*nodo_1 + 1,
102         3*nodo_1 + 2,
103         3*nodo_2,
104         3*nodo_2 + 1,
105         3*nodo_2 + 2
106     ]

```

```

107
108     for j in range(6):
109
110         for k in range(6):
111
112             K[
113                 grados[j],
114                 grados[k]
115             ] += Ki[j, k]
116
117     return K
118
119
120 def calcular_deformaciones_3D(
121     U,
122     coordenadas,
123     conectividad
124 ):
125
126     elementos = len(conectividad)
127
128     deformaciones = np.zeros(elementos)
129
130     longitudes, l, m, n = (
131         geometria_elementos_3D(
132             coordenadas,
133             conectividad
134         )
135     )
136
137     for i in range(elementos):
138
139         nodo_1 = conectividad[i][0]
140         nodo_2 = conectividad[i][1]
141
142         Le = longitudes[i]
143
144         lx = l[i]
145         ly = m[i]
146         lz = n[i]
147
148         u1 = U[3*nodo_1]
149         v1 = U[3*nodo_1 + 1]
150         w1 = U[3*nodo_1 + 2]
151
152         u2 = U[3*nodo_2]
153         v2 = U[3*nodo_2 + 1]
154         w2 = U[3*nodo_2 + 2]
155
156         deformaciones[i] = (

```

```

157         lx*(u2-u1)
158         + ly*(v2-v1)
159         + lz*(w2-w1)
160     ) / Le
161
162     return deformaciones

```

Modelo tridimensional

```

1  E = 206000
2  A = 1000
3  P = 50000
4
5  coordenadas_base = np.array([
6      [0, 0],
7      [5000, 0],
8      [10000, 0],
9      [15000, 0],
10     [20000, 0],
11     [2500, 4000],
12     [7500, 4000],
13     [12500, 4000],
14     [17500, 4000]
15 ])
16
17 coordenadas_3D = np.vstack((
18     np.column_stack((
19         coordenadas_base,
20         np.zeros(9)
21     )),
22
23     np.column_stack((
24         coordenadas_base,
25         np.full(9, 1500)
26     ))
27 ))
28
29 conectividad_base = np.array([
30     [0, 1],
31     [1, 2],
32     [2, 3],
33     [3, 4],
34
35     [5, 6],
36     [6, 7],
37     [7, 8],
38
39     [0, 5],
40     [5, 1],

```

```

41
42     [1, 6],
43     [6, 2],
44
45     [2, 7],
46     [7, 3],
47
48     [3, 8],
49     [8, 4]
50 ])
51
52 conectividad_3D = []
53
54 for barra in conectividad_base:
55
56     conectividad_3D.append([
57         barra[0],
58         barra[1]
59     ])
60
61     conectividad_3D.append([
62         barra[0] + 9,
63         barra[1] + 9
64     ])
65
66 for i in range(9):
67
68     conectividad_3D.append([
69         i,
70         i + 9
71     ])
72
73 for i in range(4):
74
75     superior = 5 + i
76
77     conectividad_3D.append([
78         i,
79         superior + 9
80     ])
81
82     conectividad_3D.append([
83         i + 9,
84         superior
85     ])
86
87     conectividad_3D.append([
88         i + 1,
89         superior + 9
90     ])

```



```

91
92     conectividad_3D.append([
93         i + 10,
94         superior
95     ])
96
97 conectividad_3D = np.array(
98     conectividad_3D
99 )
100
101 nodos_3D = len(coordenadas_3D)
102
103 longitudes_3D, l_3D, m_3D, n_3D = (
104     geometria_elementos_3D(
105         coordenadas_3D,
106         conectividad_3D
107     )
108 )
109
110 K_3D = matriz_k_3D(
111     coordenadas_3D,
112     conectividad_3D,
113     E,
114     A
115 )
116
117 F_3D = np.zeros(
118     3*nodos_3D
119 )
120
121 F_3D[3*2 + 1] = -P/2
122 F_3D[3*11 + 1] = -P/2
123
124 restringidos_3D = [
125     0, 1, 2,
126     27, 28, 29,
127     13, 40
128 ]
129
130 U_3D = resolver_sistema(
131     K_3D,
132     F_3D,
133     restringidos_3D
134 )
135
136 deformaciones_3D = (
137     calcular_deformaciones_3D(
138         U_3D,
139         coordenadas_3D,
140         conectividad_3D

```

```

141     )
142 )
143
144 tensiones_3D = calcular_tensiones(
145     deformaciones_3D,
146     E
147 )
148
149 reacciones_3D = (
150     K_3D @ U_3D - F_3D
151 )
152
153 desplazamientos_verticales_3D = (
154     U_3D[1::3]
155 )
156
157 desplazamiento_maximo_3D = np.max(
158     np.abs(
159         desplazamientos_verticales_3D
160     )
161 )
162
163 tension_maxima_3D = np.max(
164     np.abs(tensiones_3D)
165 )
166
167 longitud_maxima_3D = np.max(
168     longitudes_3D
169 )

```

Generación de figuras utilizadas en el informe

Geometría de los diseños bidimensionales

```

1  os.makedirs(
2      "Graficos",
3      exist_ok=True
4  )
5
6
7  def grafico_geometria_2D(
8      coordenadas,
9      conectividad,
10     nodo_carga,
11     apoyo_izquierdo,
12     apoyo_derecho,
13     nombre,
14     etiquetar_barras=True
15 ):

```

```

16
17     coords = coordenadas / 1000
18
19     plt.figure(
20         figsize=(11, 4.5)
21     )
22
23     for i in range(
24         len(conectividad)
25     ):
26
27         n1 = conectividad[i][0]
28         n2 = conectividad[i][1]
29
30         x = [
31             coords[n1, 0],
32             coords[n2, 0]
33         ]
34
35         y = [
36             coords[n1, 1],
37             coords[n2, 1]
38         ]
39
40         plt.plot(
41             x,
42             y,
43             linewidth=1.8
44         )
45
46         if etiquetar_barras:
47
48             xm = (
49                 x[0] + x[1]
50             ) / 2
51
52             ym = (
53                 y[0] + y[1]
54             ) / 2
55
56             plt.text(
57                 xm,
58                 ym,
59                 str(i),
60                 fontsize=8,
61                 ha="center",
62                 va="center"
63             )
64
65     plt.scatter(

```

```

66         coords[:, 0],
67         coords[:, 1],
68         s=35,
69         zorder=3
70     )
71
72     for i in range(
73         len(coords)
74     ):
75
76         plt.text(
77             coords[i, 0],
78             coords[i, 1] + 0.18,
79             f"N{i}",
80             ha="center",
81             fontsize=9
82         )
83
84     xP = coords[nodo_carga, 0]
85     yP = coords[nodo_carga, 1]
86
87     plt.annotate(
88         "P = 50 kN",
89         xy=(xP, yP),
90         xytext=(xP, yP + 1.3),
91         arrowprops=dict(
92             arrowstyle="->",
93             linewidth=2
94         ),
95         ha="center"
96     )
97
98     plt.scatter(
99         coords[
100             apoyo_izquierdo, 0
101         ],
102         coords[
103             apoyo_izquierdo, 1
104         ] - 0.25,
105         marker="^",
106         s=100
107     )
108
109     plt.scatter(
110         coords[
111             apoyo_derecho, 0
112         ],
113         coords[
114             apoyo_derecho, 1
115         ] - 0.25,

```

```

116         marker="o",
117         s=100,
118         facecolors="none"
119     )
120
121     plt.xlabel(
122         "Posicion x [m]"
123     )
124
125     plt.ylabel(
126         "Posicion y [m]"
127     )
128
129     plt.title(
130         "Geometria de la estructura"
131     )
132
133     plt.axis("equal")
134     plt.grid(alpha=0.25)
135
136     plt.tight_layout()
137
138     plt.savefig(
139         f"Graficos/{nombre}_geometria.png",
140         dpi=300,
141         bbox_inches="tight"
142     )
143
144     plt.show()

```

Distribución de tensiones en los diseños 2D

```

1  def grafico_tensiones_2D(
2      coordenadas,
3      conectividad,
4      tensiones,
5      nombre
6  ):
7
8      coords = coordenadas / 1000
9
10     segmentos = []
11
12     for barra in conectividad:
13
14         n1 = barra[0]
15         n2 = barra[1]
16
17         segmentos.append([

```

```

18         coords[n1],
19         coords[n2]
20     ])
21
22     limite = np.max(
23         np.abs(tensiones)
24     )
25
26     normalizacion = Normalize(
27         vmin=-limite,
28         vmax=limite
29     )
30
31     lineas = LineCollection(
32         segmentos,
33         cmap="coolwarm",
34         norm=normalizacion,
35         linewidths=4
36     )
37
38     lineas.set_array(tensiones)
39
40     fig, ax = plt.subplots(
41         figsize=(11, 4.5)
42     )
43
44     ax.add_collection(lineas)
45
46     ax.scatter(
47         coords[:, 0],
48         coords[:, 1],
49         s=25
50     )
51
52     barra_max = np.argmax(
53         np.abs(tensiones)
54     )
55
56     n1 = conectividad[
57         barra_max
58     ][0]
59
60     n2 = conectividad[
61         barra_max
62     ][1]
63
64     xm = (
65         coords[n1, 0]
66         + coords[n2, 0]
67     ) / 2

```

```

68
69     ym = (
70         coords[n1, 1]
71         + coords[n2, 1]
72     ) / 2
73
74     ax.annotate(
75         f"Barra {barra_max}: "
76         f"{tensiones[barra_max]:.2f} MPa",
77         xy=(xm, ym),
78         xytext=(xm, ym + 0.8),
79         arrowprops=dict(
80             arrowstyle="->"
81         ),
82         ha="center"
83     )
84
85     barra_color = fig.colorbar(
86         lineas,
87         ax=ax
88     )
89
90     barra_color.set_label(
91         "Tension axial [MPa]"
92     )
93
94     ax.set_xlim(
95         np.min(coords[:, 0]) - 1,
96         np.max(coords[:, 0]) + 1
97     )
98
99     ax.set_ylim(
100         np.min(coords[:, 1]) - 1,
101         np.max(coords[:, 1]) + 1
102     )
103
104     ax.set_xlabel(
105         "Posicion x [m]"
106     )
107
108     ax.set_ylabel(
109         "Posicion y [m]"
110     )
111
112     ax.set_title(
113         "Distribucion de tensiones axiales"
114     )
115
116     ax.set_aspect("equal")
117     ax.grid(alpha=0.25)

```

```

118
119     plt.tight_layout()
120
121     plt.savefig(
122         f"Graficos/{nombre}_tensiones.png",
123         dpi=300,
124         bbox_inches="tight"
125     )
126
127     plt.show()

```

Distribución de tensiones de la estructura 3D

```

1  def grafico_tensiones_3D(
2      coordenadas,
3      conectividad,
4      tensiones,
5      nombre
6  ):
7
8      coords = coordenadas / 1000
9
10     segmentos = []
11
12     for barra in conectividad:
13
14         n1 = barra[0]
15         n2 = barra[1]
16
17         segmentos.append([
18             coords[n1],
19             coords[n2]
20         ])
21
22     limite = np.max(
23         np.abs(tensiones)
24     )
25
26     normalizacion = Normalize(
27         vmin=-limite,
28         vmax=limite
29     )
30
31     lineas = Line3DCollection(
32         segmentos,
33         cmap="coolwarm",
34         norm=normalizacion,
35         linewidths=3
36     )

```



```

37
38     lineas.set_array(tensiones)
39
40     fig = plt.figure(
41         figsize=(10, 7)
42     )
43
44     ax = fig.add_subplot(
45         111,
46         projection="3d"
47     )
48
49     ax.add_collection3d(lineas)
50
51     ax.scatter(
52         coords[:, 0],
53         coords[:, 1],
54         coords[:, 2],
55         s=20
56     )
57
58     ax.set_xlim(
59         np.min(coords[:, 0]),
60         np.max(coords[:, 0])
61     )
62
63     ax.set_ylim(
64         np.min(coords[:, 1]),
65         np.max(coords[:, 1])
66     )
67
68     ax.set_zlim(
69         np.min(coords[:, 2]),
70         np.max(coords[:, 2])
71     )
72
73     barra_color = fig.colorbar(
74         lineas,
75         ax=ax,
76         shrink=0.7,
77         pad=0.1
78     )
79
80     barra_color.set_label(
81         "Tension axial [MPa]"
82     )
83
84     ax.set_xlabel("x [m]")
85     ax.set_ylabel("y [m]")
86     ax.set_zlabel("z [m]")

```

```

87
88     ax.set_title(
89         "Distribucion de tensiones axiales 3D"
90     )
91
92     ax.view_init(
93         elev=20,
94         azim=-60
95     )
96
97     plt.tight_layout()
98
99     plt.savefig(
100         f"Graficos/{nombre}_tensiones.png",
101         dpi=300,
102         bbox_inches="tight"
103     )
104
105     plt.show()

```

Generación de las figuras

```

1  grafico_geometria_2D(
2      coordenadas,
3      conectividad,
4      2,
5      0,
6      4,
7      "diseno_1",
8      True
9  )
10
11 grafico_tensiones_2D(
12     coordenadas,
13     conectividad,
14     tensiones,
15     "diseno_1"
16 )
17
18
19 grafico_geometria_2D(
20     coordenadas_2,
21     conectividad_2,
22     4,
23     0,
24     8,
25     "diseno_2",
26     False
27 )

```

```
28
29 grafico_tensiones_2D(
30     coordenadas_2,
31     conectividad_2,
32     tensiones_2,
33     "diseno_2"
34 )
35
36
37 grafico_tensiones_3D(
38     coordenadas_3D,
39     conectividad_3D,
40     tensiones_3D,
41     "diseno_3D"
42 )
```